Companion — BlackMatrix user guide

This module drives a BlackMatrix crosspoint server from a Stream Deck or any other Bitfocus Companion surface. It turns a fleet of Blackmagic ATEM switchers into one router panel, where every bus that takes one source at a time is a destination.

The README covers installing the module. This guide is about building a panel with it, and the places where a press can mislead an operator.

> **Before you rely on this:** BlackMatrix has **no authentication** and binds every interface. Anyone who can reach its port can re-route program on a live switcher. Keep it on a private production network only, and bear in mind that a press here re-points a live feed with no confirmation step.
>
> This module has never been loaded into Companion. It's been checked by a harness that runs its actual code against a fake crosspoint server, which isn't the same claim as "it works on a rack".
>
> This module was built with AI assistance, directed and reviewed by a human author.

Connecting

Two fields: the BlackMatrix server's host, and its port, **8533** by default. That's the crosspoint server's port, not an ATEM's.

The module keeps a WebSocket open for state and uses REST for commands, so a connection that's green but never updates is a different fault from one that won't take a command. Watch `$(blackmatrix:connection_status)` for the first and the module's log for the second.

Build an X-Y panel, not a crosspoint matrix

Get this right before you lay out a page.

A fleet of switchers has thousands of crosspoints, and one button per crosspoint gets you a spreadsheet rather than a page. So the generated presets make a **router panel** instead:

1. Press a **destination**. It turns blue. Nothing has been sent to any switcher yet.
2. Press a **source**. That crosspoint is made.

Two hundred buttons cover what would otherwise take tens of thousands, and it's how every router operator already works anyway.

The selection lives in this Companion, not in the server. Two Companions on the same fleet each have their own armed destination and never fight over a half-finished take. The flip side is that a destination armed on the rack Companion doesn't show up on the one in the gallery.

Put `Fleet → X-Y: what is selected` somewhere on the page. An armed destination that's scrolled off the visible page is how a source press ends up somewhere nobody expected.

The panel reads as a monitor wall between takes

Destination buttons carry a live label of what they're carrying right now, so the panel's worth looking at even when nothing's armed. Source buttons light **amber** when the armed destination is already taking them, so you can see the route you're about to make before you make it.

Addressing, when you build buttons by hand

Destinations and sources are `deviceId:id`:

Reference	What it is
`stage:aux.0`	Aux 1 on the switcher called `stage`
`stage:me.0.usk.1.fill`	ME 1 upstream keyer 2's fill
`stage:1`	Input 1 on `stage`

Both dropdowns take a custom value, so a variable can drive either end.

A crosspoint lives on one switcher. If you route `studio`'s source to `stage`'s aux, the module rejects it before sending anything, because an ATEM can't take another ATEM's source. If a button does nothing at all, check both halves name the same device.

Salvos fire across the fleet, and partially

A salvo sets crosspoints on every switcher in it at once. You define salvos **in BlackMatrix**, not here; this module only fires them.

If part of a salvo is refused (a locked destination, say, or a source that switcher won't take on that bus), **the rest still happens**, and the refusals are logged by name. That's the right behaviour for a live show, but it means a salvo button that lit up doesn't prove the whole salvo landed. Where a salvo has to be all or nothing, do the check before anyone presses the button, not on the button itself.

The server's refusals are worth reading rather than waving away. "Aux 1 is not available on ME 1 Program" is a switcher doing its job, not a broken module.

Locks are per IP address

BlackMatrix holds locks the way a Videohub does: by the **address** of whoever took them, not by a user or a session.

- A lock taken in BlackMatrix's browser UI on the machine running Companion has **the same owner** as one taken from these buttons. It'll unlock without complaint.
- A lock taken from any other machine belongs to somebody else, and only **force** takes it.

Lock presets are only generated for the **Outputs** (aux) section, because that's the case worth a button. The lock *action* works on any destination, so build one by hand if a keyer needs protecting.

Feedbacks, and which button they belong on

Feedback	Put it on	Says
`crosspoint`	a direct-route button	this destination is taking this source
`sourceOnSelected`	source buttons	the armed destination already has this
`destinationSelected`	destination buttons	this one is armed
`locked`	destination buttons	locked, by anyone
`deviceOnline`	a per-switcher tile	that ATEM is connected to BlackMatrix
`panelAttached`	a per-switcher tile	a hardware Videohub panel is on that switcher
`serviceUp`	one tile on the page	BlackMatrix itself is answering

`serviceUp` and `deviceOnline` answer different questions and fail separately. BlackMatrix can be perfectly happy while the switcher behind it has dropped off the network, and then the panel keeps taking presses and nothing reaches the ATEM. Put one tile of each near the other, and people go looking for the right fault.

`panelAttached` is the one people leave out and then wish they hadn't. A hardware Videohub panel on the same switcher routes the same destinations this page does, and it's got no idea this page exists.

Variables worth knowing

Variable	Value
`<switcher>_<destination>_source`	live label of what that bus is carrying
`selected_destination_label`	the armed destination's name
`selected_destination_source`	what the armed destination is currently taking
`<switcher>_videohub_port`	that switcher's Videohub port, `off` when it has none
`<switcher>_videohub_panels`	how many Videohub clients are attached to it
`locked_count`, `device_count`, `online_count`, `salvo_count`	fleet counts

Put `$(blackmatrix:stage_aux_0_source)` on a button and it becomes a monitor-wall button that labels itself.

Destination ids are flattened for variable names, so `stage:aux.0` becomes `stage_aux_0`.

When a button does nothing

Symptom	Where to look
Every button dead, no variables	Wrong host or port, or BlackMatrix is not running. `serviceUp` is the tile that says so.
Labels present but a take does nothing	The destination is locked, or the source is not legal on that bus. The module logs the server's reason.
Source press goes somewhere unexpected	A destination was still armed from earlier. Put the X-Y selection tile on the page.
Route silently refused	Source and destination name different switchers. A crosspoint cannot cross devices.
Panel disagrees with the rack	A hardware Videohub panel is routing the same destinations. `panelAttached` shows when one is on.